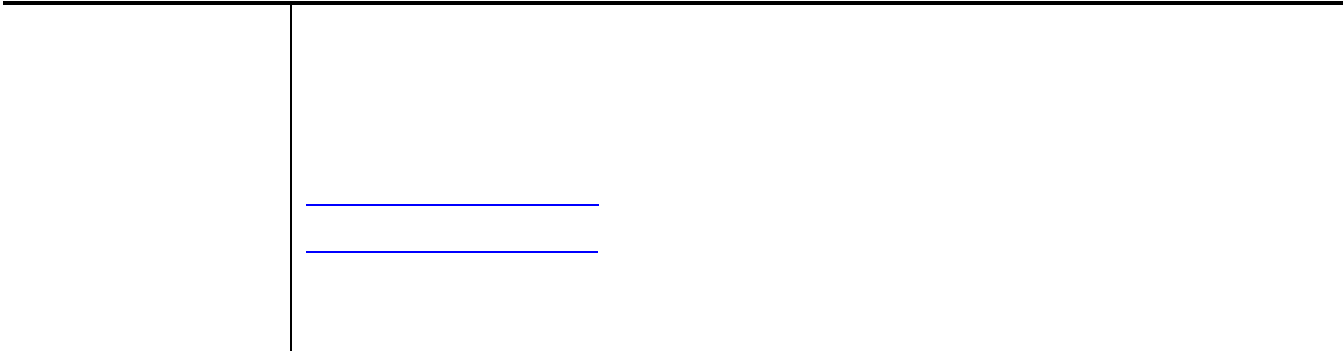


Autodesk® **Scaleform®**



.....	1
.....	2
.....	6
.....	8
.....	9
.....	15
.....	16
.....	18
.....	20
.....	21
.....	23
.....	26
.....	28
.....	29

```
scaleform.gfx.Extensions.enabled = true;
```

```
import scaleform.gfx.*;  
Extensions.enabled = true;
```

GFXSDK/Resources/AS3/CLIK

「*Edit*」 → 「*Preferences*」 → 「*ActionScript*」 → 「*ActionScript 3.0 Settings...*」 → 「*Source path*」

```
enabled:Boolean [read-write]
```

```
noInvisibleAdvance:Boolean [read-write]
```

```
public function getTopMostEntity(x :Number, y :Number, testAll:Boolean) :  
DisplayObject
```

```
public function getTopMostEntity([testAll :Boolean],  
    [mouseIndex :uint]) :DisplayObject
```

```
public function getMouseCursorType([mouseIndex :uint]) :String
```

```
mouseIndex :Number
```

- flash.ui.MouseCursor.ARROW :String = "arrow"
- flash.ui.MouseCursor.BUTTON :String = "button"
- flash.ui.MouseCursor.HAND :String = "hand"
- flash.ui.MouseCursor.IBEAM :String = "ibeam"

```
public function setMouseCursorType(cursor :String, [mouseIndex :uint]) : void
```

```
cursor :uint
```

- flash.ui.MouseCursor.ARROW :String = "arrow"
- flash.ui.MouseCursor.BUTTON :String = "button"
- flash.ui.MouseCursor.HAND :String = "hand"
- flash.ui.MouseCursor.IBEAM :String = "ibeam"

```
mouseIndex :Number
```

```
numControllers:uint [read]
```



```
public function setEdgeAAMode(dispObj:DisplayObject, mode:uint):void
```

-
-
-
-

```
dispObj :DisplayObject -
```

```
static public function getEdgeAAMode(dispObj:DisplayObject): uint
```

```
dispObj :DisplayObject
```

```
static public function setRendererString(o:DisplayObject, s:String)
```

```
import scaleform.gfx.*;
```

```
// m is a MovieClip on stage.
```

```
DisplayObjectEx.setRendererString(m, "Abc");
```

```
trace(DisplayObjectEx.getRendererString(m));
```

```
static public function setRendererFloat(o:DisplayObject, f:Number)
```

```
import scaleform.gfx.*;

// m is a MovieClip on stage.
DisplayObjectEx.setRendererFloat(m, 17.1717);
trace(DisplayObjectEx.getRendererFloat(m));

static public function disableBatching(o:DisplayObject, b:Boolean)
```

Example

```
import scaleform.gfx.*;

// m is a MovieClip on stage.

DisplayObjectEx.disableBatching(batchDisable, true);
trace(DisplayObjectEx.isBatchingDisabled(batchDisable));
```

```

package scaleform.gfx
{
    import flash.events.FocusEvent;

    public final class FocusEventEx extends FocusEvent
    {
        public var controllerIdx : uint = 0;

        public function FocusEventEx(type:String) { super(type); }
    }
}

```

```

import scaleform.gfx.*;
import flash.events.FocusEvent;

Extensions.enabled = true;

function ev(e:FocusEvent)
{
    if (e is FocusEventEx)
    {
        var ee:FocusEventEx = e as FocusEventEx;
        trace("    controllerIdx = "+ee.controllerIdx);
    }
}
stage.addEventListener(FocusEvent.MOUSE_FOCUS_CHANGE, ev);

```

```

controllerIdx :uint    [read]

```

alwaysEnableArrowKeys:Boolean [read-write]

disableFocusKeys:Boolean [read-write]

```
static public function moveFocus(keyToSimulate :String,  
                                startFromMovie:InteractiveObject = null,  
                                includeFocusEnabledChars :Boolean = false,  
                                controllerIdx : uint = 0) :InteractiveObject
```

keyToSimulate :String

startFromMovie:InteractiveObject

```
includeFocusEnabledChars :Boolean
```

```
_____  
_____  
_____  
_____
```

```
static public function findFocus(keyToSimulate :String,  
    parentMovie:DisplayObjectContainer = null,  
    loop :Boolean = false,  
    startFromMovie:InteractiveObject = null,  
    includeFocusEnabledChars :Boolean = false,  
    controllerIdx : uint = 0) :InteractiveObject
```

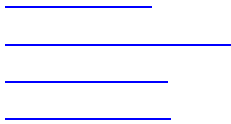
```
keyToSimulate :String
```

```
parentMovie:DisplayObjectContainer
```

```
loop :Boolean
```

startFromMovie:InteractiveObject

includeFocusEnabledChars :Boolean



static public function setFocus(obj:InteractiveObject, controllerIdx:uint = 0) : void

obj:InteractiveObject
controllerIdx : uint

static public function getFocus(controllerIdx:uint = 0) :InteractiveObject

```
controllerIdx : uint
```

```
numFocusGroups() :uint [read]
```

```
public function setFocusGroupMask(obj:InteractiveObject, mask:uint) : void
```

```
FocusManager.setControllerFocusGroup(0, 0);
```

```
FocusManager.setControllerFocusGroup(1, 1);
```

```
FocusManager.setFocusGroupMask(button1, 0x1); // bit 0 - focus group 0
```

```
FocusManager.setFocusGroupMask(movieclip2, 0x1 | 0x2); // bits 0 and 1 - focus  
// groups 0 and 1
```

```
obj:InteractiveObject
```

```
mask:uint
```

```
static public function getFocusGroupMask(obj:InteractiveObject) : uint
```

```
obj:InteractiveObject
```

```
static public function setControllerFocusGroup(controllerIdx:uint,  
                                              focusGroupId:uint) : Boolean
```

```
controllerIdx:uint  
focusGroupId:uint
```

```
static public function getControllerFocusGroup(controllerIdx:uint) : uint
```

```
controllerIndex
```

```
static public function setModalClip(mc:Sprite, controllerIdx:uint = 0) : void
```

```
controllerIdx:uint
```

```
mc:Sprite
```

```
static public function getModalClip(controllerIdx:uint = 0) :Sprite
```

```
controllerIdx
```

```
public static function supportsAnalogEvents() :Boolean
```

```

package scaleform.gfx
{
    import flash.events.Event;

    public final class GamePadAnalogEvent extends Event
    {
        public static const CHANGE:String = "gamePadAnalogChange";

        public var code : uint = 0; // See scaleform.gfx.GamePad for
                                     // valid pad codes

        public var controllerIdx : uint = 0;
        public var xvalue :Number = 0; // Normalized [-1, 1]
        public var yvalue :Number = 0; // Normalized [-1, 1]

        public function GamePadAnalogEvent(bubbles:Boolean, cancelable:Boolean,
                                           code:uint, controllerIdx:uint = 0,
                                           xvalue:Number = 0, yvalue:Number = 0)
        {
            super(GamePadAnalogEvent.CHANGE, bubbles, cancelable);
            this.code = code;
            this.controllerIdx = controllerIdx;
            this.xvalue = xvalue;
            this.yvalue = yvalue;
        }
    }
}

```

```

import scaleform.gfx.*;

```

```

trace("GamePadAnalogEvents supported?" + GamePad.supportsAnalogEvents());

```

```

function ev(e:GamePadAnalogEvent)
{
    trace("code = " + ev.code);
}

```

```
        trace("controllerIdx = " + ev.controllerIdx);
        trace("xvalue = " + ev.xvalue);
        trace("yvalue = " + ev.yvalue);
    }
    stage.addEventListener(GamePadAnalogEvent.CHANGE, ev);
```

```
code :uint
```

```
controllerIdx :uint
```

```
xvalue :Number
```

```
yvalue :Number
```

```
public function getHitTestDisable(o:InteractiveObject) :Boolean
```

o

```
public function setHitTestDisable(o:InteractiveObject, f:Boolean) : void
```

```
public function getTopmostLevel (o:InteractiveObject) :Boolean
```

```
public function setTopmostLevel(o:InteractiveObject, f:Boolean) : void
```

```

package scaleform.gfx
{
    import flash.events.KeyboardEvent;

    public final class KeyboardEventEx extends KeyboardEvent
    {
        public var controllerIdx : uint = 0;

        public function KeyboardEventEx(type:String) { super(type); }
    }
}

```

```

import scaleform.gfx.*;
import flash.events.KeyboardEvent;

```

```

Extensions.enabled = true;

```

```

function ev(e:KeyboardEvent)
{
    if (e is KeyboardEventEx)
    {
        var ee:KeyboardEventEx = e as KeyboardEventEx;
        trace("    controllerIdx = "+ee.controllerIdx);
    }
}
stage.addEventListener(KeyboardEvent.KEY_DOWN, ev);
stage.addEventListener(KeyboardEvent.KEY_UP, ev);

```

```

controllerIdx :uint    [read]

```



```

package scaleform.gfx
{
    import flash.events.Event;

    public final class MouseCursorEvent extends Event
    {
        public var cursor :String = "auto";
        public var mouseIdx : uint = 0;

        static public const CURSOR_CHANGE :String = "mouseCursorChange";

        public function MouseCursorEvent()
        {
            super("MouseCursorEvent", false, true);
        }
    }
}

```

```

Extensions.enabled = true;

```

```

function e(e:MouseCursorEvent)
{
    trace(e.type + " " + e.mouseIdx + " " + e.cursor);
    e.preventDefault();
}
stage.addEventListener(MouseCursorEvent.CURSOR_CHANGE, e);

```

cursor :String [read]

- flash.ui.MouseCursor.ARROW :String = "arrow"
- flash.ui.MouseCursor.BUTTON :String = "button"
- flash.ui.MouseCursor.HAND :String = "hand"
- flash.ui.MouseCursor.IBEAM :String = "ibeam"

mouseIdx :uint [read]

```

package scaleform.gfx
{
    import flash.events.MouseEvent;

    public final class MouseEventEx extends MouseEvent
    {
        public var mouseIdx : uint = 0;
        public var nestingIdx : uint = 0;
        public var buttonIdx : uint = 0; // LEFT_BUTTON, RIGHT_BUTTON, ...

        public static const LEFT_BUTTON : uint = 0;
        public static const RIGHT_BUTTON : uint = 1;
        public static const MIDDLE_BUTTON : uint = 2;

        public function MouseEventEx(type:String) { super(type); }
    }
}

```

```

import scaleform.gfx.*;
Extensions.enabled = true;

stage.doubleClickEnabled = true;

function ev(e:MouseEvent):void
{
    trace("!!!!EVENT." + cnt++);
    trace("    eventType      = "+e.type);
    trace("    bubbles          = "+e.bubbles);
    trace("    eventPhase         = "+e.eventPhase);
    trace("    target             = "+e.target.name);
    trace("    currentTarget      = "+e.currentTarget.name);
    if (e is MouseEventEx)
    {
        var ee:MouseEventEx = e as MouseEventEx;
        trace("    mouseIdx          = "+ee.mouseIdx);
    }
}

```

```

        trace("    nestingIdx    = "+ee.nestingIdx);
        trace("    buttonIdx     = "+ee.buttonIdx);
    }
    trace(e);
}
stage.addEventListener(MouseEvent.MOUSE_DOWN, ev);
stage.addEventListener(MouseEvent.MOUSE_UP, ev);
stage.addEventListener(MouseEvent.CLICK, ev);
stage.addEventListener(MouseEvent.DOUBLE_CLICK, ev);

```

```
buttonIdx :uint    [read]
```

```

•
•
•
•

```

```
mouseIdx :uint    [read]
```

```
nestingIdx :uint    [read]
```



```

package scaleform.gfx
{
    import flash.events.TextEvent;

    public final class TextEventEx extends TextEvent
    {
        public var controllerIdx : uint = 0;

        public function TextEventEx(type:String) { super(type); }
    }
}

```

```

import scaleform.gfx.*;
import flash.events.TextEvent;

```

```

Extensions.enabled = true;

```

```

function ev(e:TextEvent)
{
    if (e is TextEventEx)
    {
        var ee:TextEventEx = e as TextEventEx;
        trace("    controllerIdx = "+ee.controllerIdx);
    }
}
txf.addEventListener(TextEvent.TEXT_INPUT, ev);

```

```

controllerIdx :uint    [read]

```

```
public static const LINK_MOUSE_OVER:String = "linkMouseOver";  
public static const LINK_MOUSE_OUT:String = "linkMouse";
```

```
actionVerbose:Boolean [read-write]
```

```
public function getStackTrace() :String
```

```
public function getCodeFileName() :String
```



```
public function appendHtml(textField:TextField, newHtml:String) : void
```

```
textField:TextField  
newHtml:String
```

```
static public function setIMEEnabled(textField:TextField, isEnabled:Boolean): void
```

```
textField :TextField  
isEnabled :Boolean
```

```
public function setVerticalAlign(textField:TextField, valign:String) : void
```

```
public static const VALIGN_TOP:String      = "top";  
public static const VALIGN_CENTER:String  = "center";  
public static const VALIGN_BOTTOM:String  = "bottom";
```



```
textField:TextField
```

```
valign:String
```